



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

F.CSM101 Программчлалын үндсэн аргууд

Өгөгдлийн бүтэцлэлт II хэсэг

Лекц 8

*Доктор (Ph.D) **Ганбаатарын ГАНБАТ***

2023-2024 Хавар

- Полиморфизм
- Төрлийн үнэлгээ дүгнэлт
- Аюулгүй байдал
- Хуурмаг хаягаас зайлсхийх
- Хаягдал цуглуулагч



> Полиморфизм

ТОД 8.5: Нэг объект нэгээс олон төрөлтэй байж болох төрлийн системийг **полиморф (polymorphic)** гэнэ. **Полиморф** объект: төрлийн систем түүнд нэгээс олон төрөл харгалзуулсан байх.

- Нийтлэг хэлний полиморфизм
 - + нь `int×int→int`, `float×float→float`, ...; null нь `T*`; length функц нь `T[]→int`
- Хэрэглэгч нийтлэг хэл дээр полиморф объект үүсгэхгүй
 - **void** `int_sort(int A[])` ба **void** `char_sort(char C[])`-уудыг **void** `sort(<T> A[])`
- **Ad hoc полиморфизм** буюу **дахин тодорхойлолт (overloading)**
- *Универсал полиморфизм:*
 - **Параметрийн полиморфизм**
 - **Дэд төрлийн буюу хамрах (inclusion) полиморфизм.**

- 1 нэр – Олон объект: зөвхөн гадна төрхөөрөө полиморфизм (Ad hoc).
- Нэр нь ямар объект болж байгааг контекстоос нь мэднэ. Жнь:
 - “+” нэрийн хэрэглээ: int, real эсвэл string.
 - Олон функц/байгуулагч: ижил нэртэй ба парам тоо, төрлөөр ялгарах.
- Синтакс түвшин дэх хураангуй хэлбэр
- Албадлага (Coercion) биш. Тухайн хэлний тодорхойлолтоос хамаарна.

```
1 + 2
1.0 + 2.0
1 + 2.0
1.0 + 2
```

1. Дөрвөн өөр агуулгаар дахин тодорхойлно.
2. Хоёр агуулгыг (int ба real) дахин тодорхойлж, сүүлийн хоёр нь албадлагаар.
3. + нь зөвхөн бодит тоон нэмэх үйлдэл, бусад нь албадлагаар.

ТОД 8.6 (Параметрийн полиморфизм): Ерөнхий төрлийн нэгдмэл схемийг үүсгэхэд бий болох ялгаатай төрлүүдийн тоо төгсгөлөг бус байвал утга нь **универсал параметр полиморфизм**-ыг илтгэнэ.

- Универсал полиморф функц: Төрлийн мэдээлэл ашигладаггүй
 - Ерөнхий төрлийн бүх тохиолдолд жигд ажилладаг нэг кодтой
 - Полиморф төрөл: функц $\langle T \rangle [] \rightarrow \text{void}$; null утга $\langle T \rangle^*$;

```
void swap (reference <T> x, reference <T> y) {  
    <T> tmp = x;  
    x = y;  
    y = tmp;  
}
```

1-р мөр: null утга int^* төрөл
5-р мөр: тэмдэгт, 6-р мөр: бүхэл тоо.

```
int* k = null;      1  
char v, w;  
int i, j;           3  
...  
swap(v, w);         5  
swap(i, j);
```

- Параметр полиморфизм нь ерөнхий бөгөөд уян хатан. PL-д 2 хэлбэртэй:
 - **ил** (энэ хүртэл үзсэн, C++: "template", Java: "generic") болон **далд**

- Параметрийн далд полиморфизмыг зарим хэл (ML) зөвшөөрдөг
 - Программист төрөл заах шаардлагагүй
 - Төрөл шалгагч ажилладаг. Төрлийн үнэлгээ (inference) илүү тохиромжой
- Жишээ: fun – функц зарлаж буйг илэрхийлнэ

```
fun Ide (x) {return x ; }
```

- Төрлийн үнэлгээ: Ide-д $\langle T \rangle \rightarrow \langle T \rangle$ төрлийг онооно. Парам нь T бол гаралт нь T.
- (Функцийн олон аргамбаттай хослуулсан жишээ:

```
fun Comp ( f , g , x ) {return f ( g ( x ) ) ; }
```
- Comp –ийн төрлийн үнэлгээ: $(\langle S \rangle \rightarrow \langle T \rangle) \times (\langle R \rangle \rightarrow \langle S \rangle) \times \langle R \rangle \rightarrow \langle T \rangle$

- Ерөнхийдөө объект хандалгат хэлэнд байдаг.
 - Нэг объект → Төгсгөлөг бус тооны ялгаатай төрөл
 - Хамгийн ерөнхий төрлийг нь үүсгэх байдлаар бий болгоно
- Гэхдээ, хамгийн ерөнхий хэлбэрийн схемийг бүхэлд нь зөвшөөрдөггүй.
 - Төрөл хоорондын *бүтцийн* нийцтэй байдлаар хязгаарлагддаг,
 - Ө.х дэд төрөл гэсэн ойлголтыг ашигладаг.

ТОД 8.7 Ерөнхий төрлийн схемийг үүсгэхдээ оногдсон төрлийн дэд төрлүүдийг параметрийн оронд авахад бий болох ялгаатай төрөлтэй төгсгөлөг бус байвал

утга нь **дэд төрөл (эсвэл зааглагдсан) полиморфизмыг** харуулдаг

$\forall T < : D . T \rightarrow \text{void}$

- D-ийн дурын дэд төрлийн утганд хэрэглэнэ ($X < : Y$ бол "X нь Y-ийн дэд төрөл")

```
int f(int n){return n+1;}
```

Төрлийн үнэлгээ

- Төрөл үнэлгээ (ML хэл): Төрөл шалгалтаас илүү боловсронгуй
 - Бүрдэл хэсгүүдийн төрөл ил зарлалтгүй байхад төрлийг оноох процесс.

```
fun f(n){return n+1;}
```

- f нь $\text{int} \rightarrow \text{int}$ төрөлтэй гэдгийг автоматаар гарган авах
- Үнэлгээний алгоритм
 - Төрөл шалгалт шиг синтакс мод дээр навчнаас эхлэн ажилладаг боловч атомик илэрхийлэлд (n) зориулсан төрлийг шууд тогтоох боломжгүй.
 - Иймд 'а' (ML тэмдэглэгээ) шиг *төрөл хувьсагч* авч үздэг.
 - Модонд дахин нэвтэрч, контекст мэдээллийг ашиглан төрөл хувьсагчийн зарим шаардлагыг цуглуулна.
 - Жишээний хувьд 1 ба "+"-ийн төрлөөс 'а' = int гэж гаргаж авдаг.

Функцийн хамгийн ерөнхий төрлийг тогтоох чадвартай

1. Синтакс модны зангилаа бүрт төрөл онооно.
 - Predefined нэр, тогтмол бүрт симбол хүснэгтийн төрлийг ашиглана.
 - Шинэ (жнь, программист тодорхойлсон) танигч, нийлмэл илэрхийлэл бүрт (дотоод зангилаа) төрөл хувьсагч (илэрхийлэл/нэр бүрт шинэ) ашиглана;
2. Дахин синтакс модоор явж, дотоод зангилаа бүрт төрөл хоорондын шаардлагыг (=) үүсгэнэ.
 - 'a төрөлтэй f функц тэмдэгийг 'b төрөлтэй v аргументад хэрэглэвэл, f нь үнэхээр функц бөгөөд v-тэй ижил төрөлтэй байх ёстой тул $'a = 'b \rightarrow 'c$ гэсэн шаардлага үүсгэгдэнэ ('c нь шинэ төрөл хувьсагч).
3. Нэгтгэх (unification) алгоритмыг ашиглан цуглуулсан шаардлагуудыг шийднэ (хүчирхэг, гэхдээ ойлголтын хувьд энгийн).
 - `fun g(n) {return n;}` нь $'a \rightarrow 'a$ төрөлтэй. Хамгийн ерөнхий төрөл нь төрөл хувьсагч агуулсан илэрхийлэл бол полиморф байна.

PL-ний аюулгүй байдал

UNSAFE:

- From: presence of unions
- From: explicit deallocation of memory

Unsafe

- Төрлийн систем нь арга зүй шаарддаг, Төрөл шалгалтыг алгасдаг
- C, C++ болон уг бүлгийн хэлүүд

Locally unsafe

- Төрлийн системийн сайн зохицуулалттай, Төрөл шалгадаг
- Алдаатай программ бичигдэх боломжтой дэд хэсгүүдтэй
- Algol, Pascal, Ada болон тэдний залгамжлагч хэлүүд

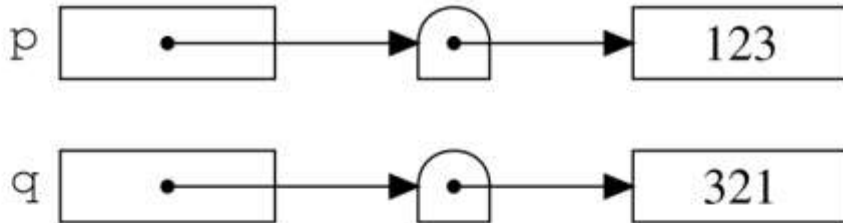
Safe

- Төрлийн зөрчлөөс үүдэлтэй далд алдаагүйг теоремоор баталдаг
- Төрлийн динамик (Lisp, Scheme), статик (ML) шалгалттай
- Төрлийн статик + runtime (Java) шалгалттай

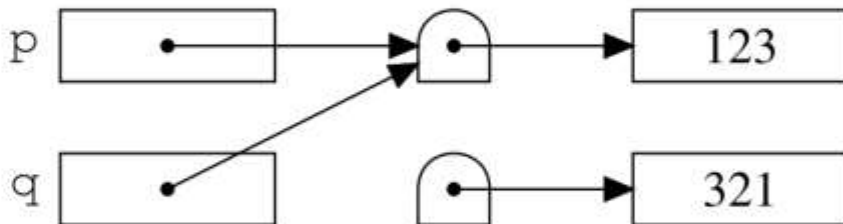
#> Хуурмаг хаягны цэвэрлэгээ

- Хуурмаг заагчийг динамикаар хориглох талаар хийсвэр машинд хамраарагдах механизмуудын асуудлыг авч үзнэ.
- **Томбстоун (Tombstone):**
 - Хийсвэр машин хуурмаг хаягийг дахин хаяглах бүрийг бүртгэж чадна.
- **Locks and Keys:**
 - Энэ нь томбстоуны хуримтлагдсан асуудлаас ангид байх механизмыг ашиглан овоолго доторх хуурмаг хаягын асуудлыг шийддэг

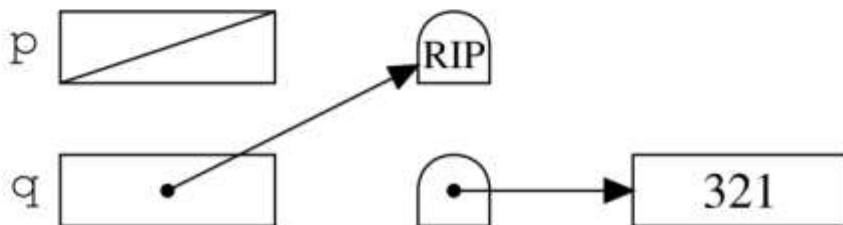
```
p=malloc();  
q=malloc();  
*p=123;  
*q=321;
```



```
q=p;
```



```
free(p);
```

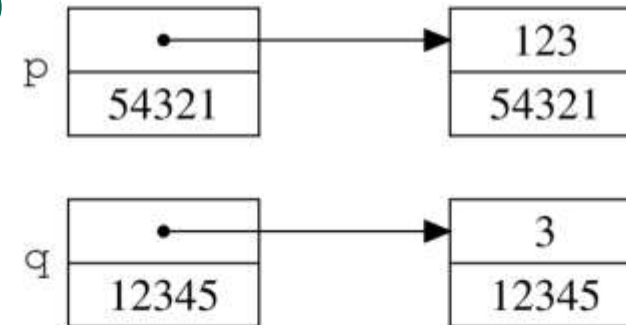


- Объектийг heap/стэкт хуваарилах үед нэмэлтээр хуваарилдаг үг
- Объектыг чөлөөлөх үед TS-д тусгай тэмдэглэгээ (RIP) хийнэ
 - for: Физик машины хаяг хамгаалах механизм
- Cemetery: Ижил хэмжээтэй тул хялбар удирдана.
- Хугацааны зардал
 - Үүсгэх, шалгах, 2 түвшний заагч
- C/O-н зардал
 - Жижиг объект олон бол их
 - Устгагдахгүй – Хаяг тоолуур хэрэглэнэ.

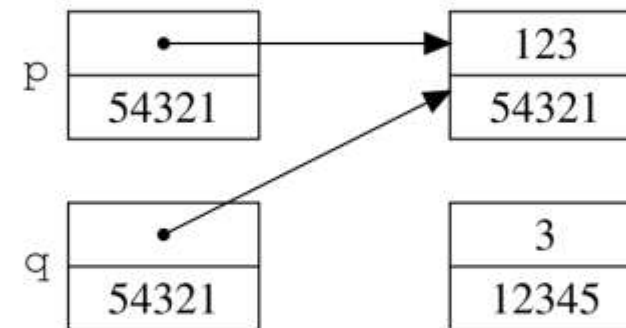
ХУУРМАГ ХАЯГНЫ ЦЭВЭРЛЭГЭЭ» Түгжээ ба түлхүүр

- Объектийг heap хуваарилах үед нэмэлтээр хуваарилдаг 2 үг: *lock & key*
- Нэг заагчийг нөгөөд олгох үед уг хосыг онооно.
- Заагчийг дахин хаягжуулах бүрт түлхүүрийг түгжээтэй нь шалгана.
- Объект чөлөөлөхөд key-г 0 болгож бүх түгжээ алдаа өгөхөөр болно
- Хугацааны зардал
 - Үүсгэх, утга олгох, key-г шалгах
- C/O-н зардал
 - Заагч бүртэй нэмэлт үг холбох TS-ээс ч их

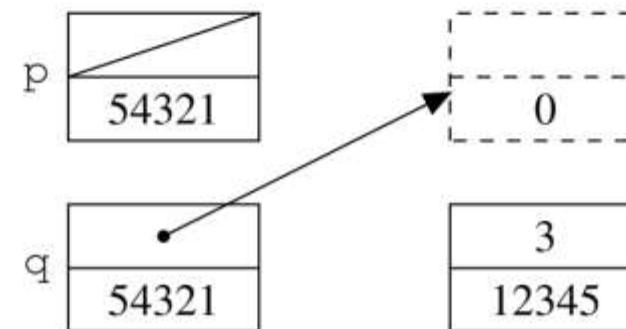
```
p=malloc();  
q=malloc();  
*p=123;  
*q=3;
```



```
q=p;
```



```
free(p);
```

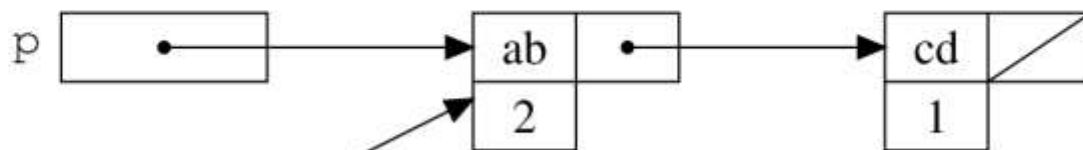
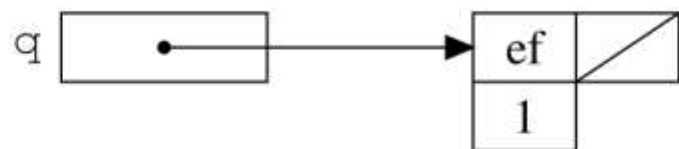
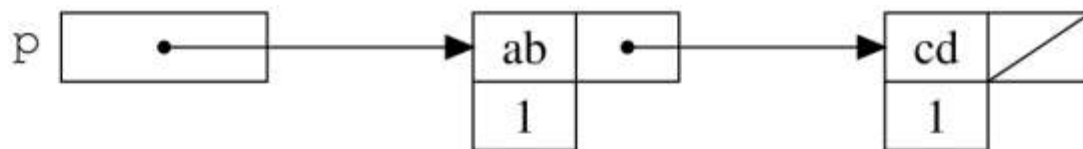


possibly
reallocated

Хаягдал цуглуулагч (garbage collector)

- СО-г ил чөлөөлөхгүй хэлний хувьд овоолго дээр нөөцлөөд, хэрэглэхээ больсон СО-г автоматаар буцаан авах механизм шаардлагатай.
 - 1960s, LISP; Java – хүчирхэг GC
- Хаягдал цуглуулалгчийн алхамууд
 1. Оршин байгаа, хэрэглэхээ больсон объектуудыг ялгах (хагдал илрүүлэх);
 2. Хэрэглэхээ больсон объектуудыг цуглуулж, тэдний зайг программ дахин ашиглах боломжтой
- Хэрэглэхээ больсон объектуудыг илрүүлэх байдлаар ангилж болно:
 - **Хаяг тоолуур (reference counting),**
 - **marking**
 - **copying.**

- Объектыг хэрэглэхээ больсон эсэхийг мэдэх хамгийн энгийн арга бол түүнд заагч байгаа эсэхийг хянах



q=p;

- Программист хандахгүй, хийсвэр машин хариуцна
- Тоолуур 0 болоход холбоотой объектыг салгаж, чөлөөт жагсаалтад буцаана
- Объектын дотор заагчтай бол рекурсив байдлаар цуглуулдаг.
- Гол дутагдал:** Цикл бол ажиллахгүй
- Зардал:** программын биелүүлсэн ажлын тоотой пропорциональ
- Python-ны CPython VM хэрэгжилт

Тэмдэглээд шүүрдэх (Mark and Sweep)

- **Тэмдэглэх (Mark).** Овоолгын бүх объектоор тойрон "хэрэглээнд байхгүй", "хэрэглээнд байгаа" гэж тэмдэглэнэ.
- **Шүүрдэх (Sweep).** "хэрэглээнд байгаа" блокуудыг үлдээж, "хэрэглээнд байхгүй" блокуудыг чөлөөт жагсаалт руу буцаана.
- Овоолгын сул зай дуусах үед л ажиллана (Хаяг тоолуур байнга ажиллана)
- Сул тал
 - Гадаад хэсэгчлэл: оршин байгаа болон хэрэглэхээ больсон объектууд овоолгод дурын байдлаар байрладаг. олон жижиг блокт сул байгаа хэдий ч том объект хуваарилахад төвөгтэй болгодог
 - Үр ашиг: Ашигласан болон сул зайны хувь хэмжээнээс үл хамааран овоолгын нийт урттай пропорциональ цаг хугацаа шаардагдана
 - Хаягийн байршил: "Хэрэглээнд байгаа" объектууд байрандаа үлддэг. Зэргэлдээ хоосон байрлалд шинэ объект байрлуулах боломжтой.

Тэмдэглээд нягтруулах(Mark and Compact)

- Хэсэгчлэлээс зайлсхийхийн тулд “шүүрдэх”-ийн оронд нягтруулж болно.
 - Оршин буй объектыг хажуу тийш нь зөөх: Чөлөөт СО-н үргэлжилсэн блок
 - Нягтруулах ажлыг овоолгын дээр шугаман шилжилт хийж, тааралдсан оршин байгаа блок бүрийг хөдөлгөж, өмнөх блоктой зэрэгцүүлнэ.
- Овоолгын дээр нэгээс олон шилжих шаардлагатай (хугацаа нь овоолгын хэмжээтэй пропорциональ).
 - Эхнийх нь оршин байгаа блокуудын авах шинэ байрлалыг тооцоолох;
 - Хоёр дахь нь дотоод заагчийг шинэчилж,
 - Гурав дахь нь объектыг хөдөлгөдөг. Өмнөхөөс ч илүү зардалтай
- Хаягийн байршилд оновчтой нөлөө үзүүлдэг. Энэ нь чөлөөт жагсаалтыг нэг блок гэж үзэхийг дэмждэг. Шинэ объектуудыг чөлөөт жагсаалтаас энгийн заагч хасах замаар хуваарилдаг.

- "хаягдал"-д тэмдэглэгээ хийх үе шат байхгүй.
 - Зөвхөн оршин байгаа блокуудыг хуулбарлах, нягтруулах ажил байдаг.
 - Ил тэмдэглэгээний үе шат байхгүй, орон зайг огт өөр аргаар зохицуулах нь түүний зардлыг тэмдэглэгээнд суурилсан алгоритмаас эрс өөр болгодог.
- Овоолгыг ижил хэмжээтэй хоёр хэсэгт хуваагддаг.
 - Хэвийн биелэлтийн үед зөвхөн нэг нь ашиглагдана.
 - Санах ой нь хагас орон зайн нэг талд байрладаг, Чөлөөт СО нь хуваарилалт бүрт багасдаг ганц үргэлжилсэн блоктой.
 - Хуваарилалт нь онцгой үр ашигтай бөгөөд хэсэгчлэл байхгүй.
- Санах ой дуусах үед хаягдал цуглуулагчийг дууддаг.
 - Стек (the root set) дэх заагчуудаас эхлээд одоогийн хагаст (fromspace) байгаа бүтцийн холбоост жагсаалтанаар зочилж, бүтцийг нөгөө хагаст (tospace) нэг нэгээр нь хуулах бөгөөд сүүлийнх дээр нь нягтруулдаг.
 - Дуусахад хоёр орон зайн үүрэг солигдож, биелэлт user программд буцдаг.



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

АНХААРАЛ ТАВЬСАНД БАЯРЛАЛАА